



1ACC0235 Procesamiento de Imágenes
Trabajo Parcial

Tema:

Modelo de detección de vehículos y objetos en calles peruanas

Profesor:

Javier Ulises Rosales Huamanchumo

Sección:

3238

Integrantes:

Ambrocio Tantalean, Ricardo Victor (U202315214)
León Quispe, Linda Libertad (U202322089)
Quispe Zavaleta, Jose Alberto (U20221C464)
Rivera Contreras, Pamela Ivonne (U202216662)

TABLA DE CONTENIDO

1. Introducción y Objetivos	3
2. Metodología CRISP-DM	3
Business Understanding	3
Data Understanding	4
Data Preparation	5
Modeling	5
Evaluation	7
Deployment	11
3. Desarrollo del Software	13
Diagrama de componentes y dependencias	13
Guía de instalación, configuración y uso (README)	13
4. Resultados y Conclusiones	15
Discusión de los resultados frente a la métrica objetivo	15
Limitaciones identificadas	15
Propuestas de Trabajo futuro	16
5. Anexos	16

1. Introducción y Objetivos

Hoy en día el área de la autoconducción ha tenido un auge muy grande. Diversas empresas han creado sus propios modelos de detección de objetos en carreteras, el cual es el primer paso para lograr que un auto conduzca por su cuenta sin necesidad de un conductor humano.

Sin embargo, estos modelos están entrenados en entornos de conducción perfectos, o casi perfectos, como las carreteras de Alemania o Estados Unidos. Esto hace que al exponer al vehículo autónomo a calles con tráfico caótico como el nuestro falle estrepitosamente. El modelo no brinda un reconocimiento acertado ya que no reconoce los objetos que pasan frente a él mientras que para nosotros son comunes.

Por lo tanto, creemos que es necesario contar con un modelo de detección vial que pueda entender y desenvolverse correctamente en distintos entornos. Tanto en entornos con alta educación cívica como aquellos en los cuales aún la educación mencionada y el orden vehicular están en condiciones precarias.

El objetivo principal del proyecto es crear un modelo de detección vial que funcione correctamente en el tráfico peruano. Este modelo deberá de ser capaz de detectar los diversos objetos que día a día se presentan en las calles de nuestro país y que son casi inexistentes en otros países de primer mundo como mototaxis, desmonte, combis, etc.

2. Metodología CRISP-DM

Business Understanding

El problema que atiende este proyecto es la ausencia de modelos de detección vial entrenados para el contexto peruano. Los sistemas existentes de conducción autónoma han sido desarrollados en entornos con tráfico normado, lo que los hace incapaces de reconocer objetos viales propios del Perú como mototaxis, combis, carretillas o desmonte en vía pública.

La métrica objetivo principal es el $mAP@0.5$ (mean Average Precision con umbral IoU de 0.5), estándar en tareas de detección de objetos. Se establece como criterio de éxito que el modelo alcance un $mAP@0.5 \geq 0.65$ en el conjunto de validación, superando como línea base un detector genérico preentrenado en COCO sin fine-tuning aplicado al mismo dataset. Como métricas complementarias se considerarán un recall ≥ 0.60 para las clases críticas (mototaxi, peatón, vehículo) y un tiempo de inferencia menor a 50 ms por frame.

Data Understanding

Para el análisis exploratorio se utilizó el dataset robusto-1_extended de Artificio Org, disponible en Hugging Face. Este cuenta con 203 videos de 5 segundos grabados a 10 fps en primera persona desde un vehículo en movimiento, en tres ciudades del Perú: Lima, Cusco y Cajamarca. Todos los frames presentan resolución uniforme de 1920×1080 píxeles con 3 canales RGB.

Dado que el dataset está compuesto por videos, se extrajo un frame representativo por video, correspondiente al segundo 2.5 de cada grabación, evitando así frames negros de transición al inicio o al final. Esto resultó en 203 frames sobre los cuales se realizó el análisis.

Los principales hallazgos del EDA fueron los siguientes:

- **Distribución de Iluminación:** El histograma promedio en escala de grises muestra una distribución en campana ancha centrada entre las intensidades 80 y 110, reflejo de la combinación de asfalto, edificaciones y vegetación presente en escenas viales urbanas. Se detectó además un pico pronunciado cercano a la intensidad 250, indicativo de sobreexposición frecuente asociada a grabaciones diurnas.
- **Métricas de Brillo y Contraste:** El análisis de brillo por frame arrojó una media de 118.73/255, con la mayoría de frames concentrados entre 110 y 150, confirmando el predominio de condiciones diurnas. La desviación estándar del brillo entre frames fue de 21.48, valor moderado que indica variabilidad sin casos extremos. El frame más oscuro registró un brillo de 39/255 y el más brillante de 155.8/255. Respecto al contraste, la media fue de 57.93/255, valor favorable para la detección de objetos ya que implica suficiente diferencia de intensidad entre los elementos de interés y su entorno.
- **Diagnóstico de Sobreexposición:** Estableciendo un umbral en píxeles mayor igual a 245, se determinó que en promedio el 1.64% de los píxeles del dataset están quemados. Se identificaron 20 frames con más del 5% de pérdida de información y 9 con más del 10%, destacando el frame Robusto 1_16.mp4 como el caso más crítico con un 20.89% de píxeles quemados.
- **Desbalance Crítico de Iluminación:** Se identificó una severa asimetría en las condiciones temporales del dataset, registrándose un 88.7% de frames diurnos reales (180 imágenes) frente a un escaso 11.3% de frames nocturnos u oscuros.

(23 imágenes). Este marcado desbalance lumínico representa un riesgo directo de sesgo para el entrenamiento del modelo, ya que la escasez de muestras nocturnas impediría que la red neuronal aprenda a generalizar correctamente en escenarios de baja visibilidad, provocando una inminente caída en las métricas de rendimiento global.

Data Preparation

Como respuesta a las limitaciones de calidad y el desbalance identificados en el EDA, el procesamiento de los datos se estructuró en dos decisiones principales:

- Filtro por Desbalance Lumínico (Exclusión Nocturna): Debido a que el dataset presentaba un sesgo crítico hacia las grabaciones diurnas (180 frames, 88.7%) frente a una presencia insuficiente de frames nocturnos/oscuros (23 frames, 11.3%), se decidió eliminar por completo la data nocturna real. Esta exclusión evitó contaminar el entrenamiento con muestras asimétricas que habrían provocado la caída de las métricas del modelo. Tras el filtrado, el dataset diurno se estabilizó con un brillo promedio de 123.99/255 y una desviación estándar optimizada de 16.08.
- Transformación Lineal: Utilizando parámetros de $a=0.75$ y $b=20$, logró reducir los píxeles quemados del peor caso al 0.00%, pero degradó el contraste global (cayendo de 73.98 a 55.39) al apagar la imagen de forma uniforme.
- CLAHE (Contrast Limited Adaptive Histogram Equalization): Aplicado de forma adaptativa con $\text{clipLimit}=2.0$ y $\text{tileGridSize}=(8,8)$ en el espacio HSV. CLAHE demostró ser la solución óptima: a nivel global redujo el promedio de píxeles quemados de 1.64% a solo 0.34% y elevó el contraste neto del dataset en +1.07 (std).

Modeling

En esta fase se detalla la selección de la arquitectura, la configuración de los hiperparámetros y la estrategia de entrenamiento empleada para el desarrollo del modelo de visión artificial.

1. Selección y Justificación de la arquitectura

Para resolver la tarea de detección de objetos (*Object Detection*), se seleccionó la arquitectura Faster R-CNN, la cual opera como un detector de dos etapas (*Two-Stage Detector*). Como extractor base de características (*backbone*), se

integró la red ResNet-50 en combinación con una Red de Pirámide de Características (FPN - *Feature Pyramid Network*).

La elección de un detector de dos etapas se fundamenta en su capacidad para desacoplar el proceso de localización del de clasificación, lo que ofrece una precisión superior en escenarios urbanos complejos frente a los enfoques de una sola etapa. Mientras que los modelos de una etapa procesan la imagen mediante una cuadrícula rígida y directa que suele fallar ante oclusiones severas u objetos superpuestos, Faster R-CNN emplea una Red de Propuesta de Región (RPN) para aislar de forma independiente propuestas densas de regiones candidatas mediante la evaluación de *Anchor Boxes*. Esta característica prioriza el *Recall* y la exactitud de las delimitaciones en calles con alta densidad vehicular y peatonal.

El flujo de procesamiento de la arquitectura implementada se compone de los siguientes elementos clave:

- **Backbone (ResNet-50 + FPN):** Se encarga de la extracción de mapas de características multiescala, permitiendo que el sistema reconozca eficazmente objetos de diferentes dimensiones espaciales, desde vehículos cercanos hasta peatones o mototaxis lejanas.
- **RPN (Region Proposal Network):** Analiza los mapas de características generados para determinar de manera probabilística las zonas de la imagen con mayor densidad de presencia de objetos candidatos.
- **RoIAlign:** Extrae y escala las características exactas de cada propuesta de región a un tamaño uniforme de 7x7 píxeles, evitando los procesos de cuantización que degradan la localización espacial en la imagen original.
- **Fast R-CNN:** Constituye la etapa final donde las capas completamente conectadas (*Fully Connected*) asocian y combinan simultáneamente toda la información de la región para determinar la clase del objeto detectado y refinar la geometría final de la caja delimitadora (*Bounding Box*).

2. Configuración de Hiperparámetros

El régimen de entrenamiento se estructuró detalladamente con el objetivo de asegurar la convergencia de las funciones de pérdida y mitigar el riesgo de sobreajuste (*overfitting*). Las especificaciones de los hiperparámetros definitivos para el pipeline de ejecución son:

- **Número de Épocas:** Se estableció un ciclo de 100 épocas de entrenamiento para permitir un aprendizaje profundo de los patrones visuales.
- **Optimizador:** Se utilizó el algoritmo de Descenso de Gradiente Estocástico (SGD) con un parámetro de *momentum* fijado en 0.9, garantizando estabilidad en la navegación sobre superficies de pérdida complejas.
- **Planificador de Tasa de Aprendizaje (Scheduler):** Se implementó un decaimiento dinámico basado en *Cosine Annealing LR*. Este planificador disminuye la tasa de aprendizaje de forma suave y sinusoidal a lo largo de las épocas, facilitando que el modelo se asiente en un mínimo global óptimo y evite oscilaciones abruptas al final del entrenamiento.

3. Estrategia de Entrenamiento

Debido a que el dataset del proyecto cuenta con un volumen optimizado pero acotado para realizar un entrenamiento desde cero, se aplicó una estrategia de transferencia de aprendizaje (Transfer Learning) utilizando pesos preentrenados del dataset ImageNet, complementada con un proceso de Fine-Tuning diferencial.

Bajo este enfoque, las primeras capas de la arquitectura ResNet-50 se mantuvieron totalmente congeladas, puesto que sus filtros ya se encuentran optimizados para la extracción de características primitivas de bajo nivel como bordes, texturas y esquinas genéricas. Por el contrario, se desbloquearon y reentrenaron específicamente las capas profundas conocidas como layer3 y layer4 del backbone, además de la RPN y las capas lineales finales. Esta configuración diferencial permitió que la red adaptara sus representaciones abstractas de alto nivel para resolver la severa ambigüedad visual existente entre siluetas de vehículos similares dentro del entorno urbano, como la distinción crítica entre las clases Carro y Mototaxi, sin destruir el conocimiento previo del modelo ni provocar el desvanecimiento del gradiente.

Evaluation

En esta sección se analizan cuantitativa y cualitativamente los resultados obtenidos por el modelo en el conjunto de prueba (*Test*) tras las 100 épocas de entrenamiento, evaluando el rendimiento general, el comportamiento individual de las clases, el comportamiento interclase cruzado y el análisis detallados de los errores detectados.

1. Métricas de Rendimiento Global y por Clase

La evaluación del modelo se rigió bajo los estándares internacionales de las competencias Pascal VOC y COCO (Common Objects in Context), utilizando la métrica de Precisión Promedio Media (mAP) bajo diferentes umbrales de Intersección sobre la Unión (IoU). Tras el rediseño arquitectónico y la consolidación de etiquetas, el modelo alcanzó un mAP@50 de 68.37% en el conjunto de prueba (Test). Este resultado demuestra una alta viabilidad para la detección en el entorno urbano de Lima, reflejando que el modelo es altamente permisivo y eficaz al localizar objetos con un solapamiento del 50% ($\text{IoU} \geq 0.50$). Al evaluar bajo criterios más estrictos, como el mAP@75 ($\text{IoU} \geq 0.75$) y el promedio exhaustivo mAP@50:95, se observó una degradación natural en la métrica debido a la alta variabilidad en el posicionamiento y la geometría exacta de las cajas delimitadoras inducida por la deformación de perspectiva en las cámaras de vigilancia urbana.

[TEST] Métricas de Evaluación			
mAP@50	:	0.6837	
mAP@75	:	0.3825	
mAP@50:95	:	0.5331	
F1 macro	:	0.6466	
F1 por clase (score_threshold=0.5):			
Carro	P=0.793	R=0.770	F1=0.781
Mototaxi	P=0.464	R=0.912	F1=0.615
Persona	P=0.516	R=0.574	F1=0.543
AP@[.50:.75] por clase (SIN score_threshold, ranking completo):			
Carro	AP=0.6418		
Mototaxi	AP=0.6429		
Persona	AP=0.3147		

[INFO] Mejor mAP@50 (val):	0.7283
[INFO] mAP@50 (test):	0.6837
[INFO] F1 macro (test):	0.6466

Para obtener una visión balanceada del rendimiento interclase, se evaluó el F1-score macro, logrando un 64.66%. El desglose detallado de las métricas de precisión (P), exhaustividad (R) y puntuación F1 (F1) bajo un umbral de confianza (score threshold) de 0.5 revela el comportamiento del modelo para cada superclase:

- **Carro (F1 = 0.781)**: Es la clase más robusta y equilibrada del sistema, logrando una precisión de 79.3% y un Recall de 77.0%. Su precisión promedio independiente (AP) se situó en 64.18%, consolidándose como la categoría con mayor estabilidad geométrica y abundancia de muestras.
- **Mototaxi (F1 = 0.615)**: Presenta un comportamiento asimétrico muy marcado, registrando el Recall más alto del sistema con un 91.2%,

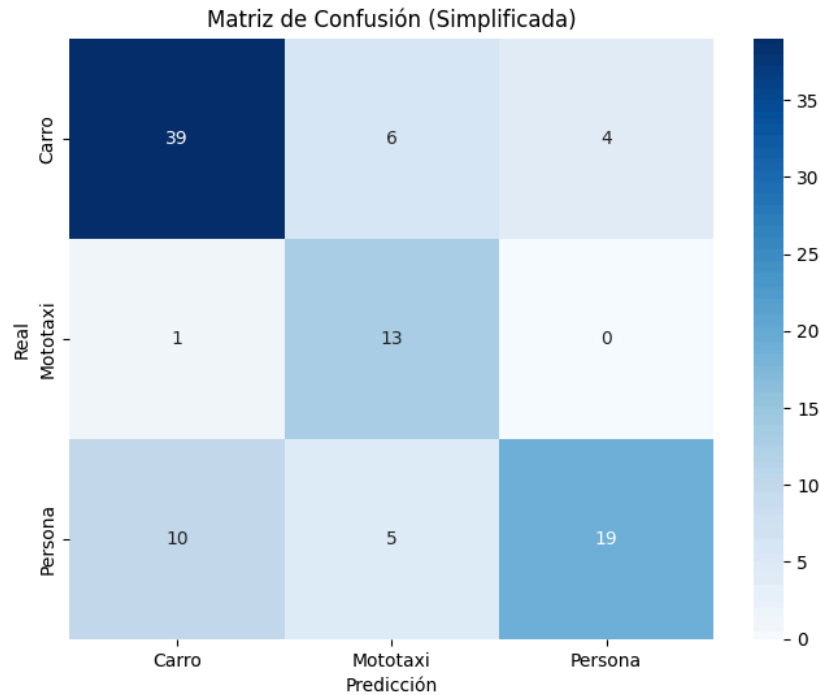
contrastado con una precisión de 46.4%. Esto indica que el modelo es altamente sensible para localizar mototaxis (encuentra casi todas las presentes en la escena), pero tiende a generar un volumen considerable de falsos positivos al etiquetar erróneamente otros objetos como tales. Su AP final fue de 64.29%.

- **Persona (F1 = 0.543):** Registró el menor desempeño del pipeline, con una precisión de 51.6%, un Recall de 57.4% y un AP de 31.47%. Este comportamiento responde directamente a los desafíos de escala y oclusión inherentes a los peatones en entornos públicos abiertos.

2. Matriz de Confusión

Para evaluar el desempeño individual y el direccionamiento de las predicciones erróneas, se analizó la matriz de confusión del conjunto de prueba. El comportamiento cruzado de las clasificaciones se distribuyó de la siguiente manera:

- **Clase Carro:** De las instancias evaluadas, 39 fueron clasificadas correctamente. Sin embargo, el modelo desvió 6 instancias hacia la clase Mototaxi y 4 hacia la clase Persona.
- **Clase Mototaxi:** Registró una alta efectividad de acierto directo con 13 instancias correctamente identificadas, desviando únicamente 1 instancia hacia la categoría de Carro y ninguna hacia Persona.
- **Clase Persona:** Mostró la mayor dispersión en la matriz, logrando clasificar correctamente 19 instancias, pero sufriendo una fuerte confusión al etiquetar 10 de ellas como Carro y 5 como Mototaxi.



3. Análisis de Errores

A partir de los resultados de la matriz de confusión y de la revisión visual de las predicciones incorrectas en el conjunto de prueba, se identificaron tres causas principales que explican los errores del modelo:

- **Falsos Positivos en la Clase Mototaxi por Ambigüedad**

Geométrica: La baja precisión de la clase Mototaxi (46.4%) se explica directamente por las transferencias erróneas observadas en la matriz, donde el modelo clasificó 6 carros y 5 personas como si fuesen mototaxis. Debido a la naturaleza del entorno urbano de Lima, la geometría angular de las mototaxis y la presencia de componentes expuestos (como carpas, barras de protección o parabrisas pequeños) provocan que el clasificador final de Faster R-CNN confunda fragmentos de vehículos mayores o peatones que transitan en extrema proximidad como si constituyeran una unidad de mototaxi independiente.

- **Confusión Bidireccional en Puntos de Fuga (Efecto**

Carro-Persona): La matriz evidencia una confusión crítica donde 10 personas fueron etiquetadas como Carro y 4 carros como Persona. Este error se concentra en los puntos de fuga de las avenidas y paraderos. Al reducirse drásticamente la resolución espacial por la distancia al lente, los peatones que caminan agrupados pierden su definición morfológica individual; la Red de Propuestas de Región (RPN) genera una caja

envolvente densa que el clasificador final interpreta como la silueta cuadrangular de un vehículo menor, provocando un falso positivo estructural.

- **Falsos Negativos y Desafíos de Escala en la Clase Persona:** Las 19 instancias logradas en la clase Persona demuestran que el modelo es capaz de reconocer peatones cuando estos presentan suficiente resolución espacial. Sin embargo, la dispersión de 10 instancias hacia Carro y 5 hacia Mototaxi revela que el verdadero problema con los peatones no es que el modelo los ignore por completo (falsos negativos absolutos), sino que los clasifica erróneamente debido a la pérdida de morfología. Al encontrarse a grandes distancias o en condiciones de baja iluminación, la red detecta la presencia de un objeto, pero las capas profundas (layer3 y layer4) de la ResNet-50 no logran extraer características lo suficientemente finas para discriminar la silueta humana, asignándolos erróneamente a las categorías vehiculares adyacentes por un tema de proximidad en el espacio de características.

Deployment

En esta fase final se describe la transición del modelo entrenado desde el entorno de laboratorio (Jupyter Notebook) hacia un entorno de producción accesible para los usuarios finales, detallando la arquitectura de software y los componentes de la aplicación.

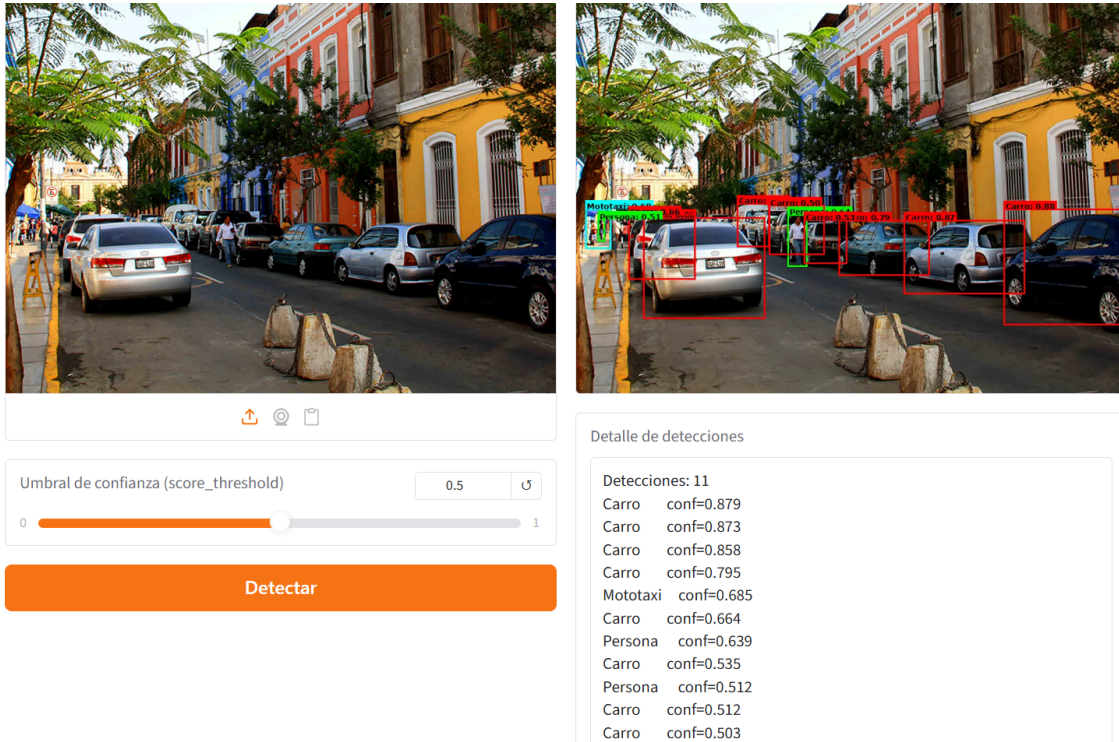
1. Descripción de la Aplicación e Interfaz de Usuario

El despliegue del modelo se consolidó mediante una aplicación web interactiva orientada a la verificación visual en tiempo real. La interfaz de usuario fue desarrollada utilizando el framework Gradio, una librería de Python especializada en la creación de prototipos rápidos y accesibles para modelos de Machine Learning.

La aplicación consta de un diseño de tres componentes principales:

- **Panel de Entrada (Input):** Un contenedor de arrastrar y soltar (drag-and-drop) que permite al usuario cargar imágenes urbanas en formatos estándar (JPEG, PNG).
- **Panel de Visualización de Inferencias (Output Visual):** Un lienzo interactivo que renderiza la imagen original superponiendo las cajas delimitadoras (Bounding Boxes) calculadas por Faster R-CNN, etiquetando dinámicamente cada objeto según sus tres superclases (Carro, Mototaxi, Persona) utilizando colores diferenciados.

- **Módulo de Análítica Teórica:** Un componente secundario en formato de tabla de datos que desglosa de manera estructurada el conteo total de objetos identificados en la escena junto con sus respectivos niveles de confianza o scores probabilísticos.



2. Arquitectura de Software y Despliegue en la Nube

Para garantizar la disponibilidad y el acceso remoto a la solución sin depender de recursos locales, el pipeline completo fue alojado en la plataforma de computación en la nube Hugging Face Spaces (disponible públicamente bajo el repositorio del proyecto).

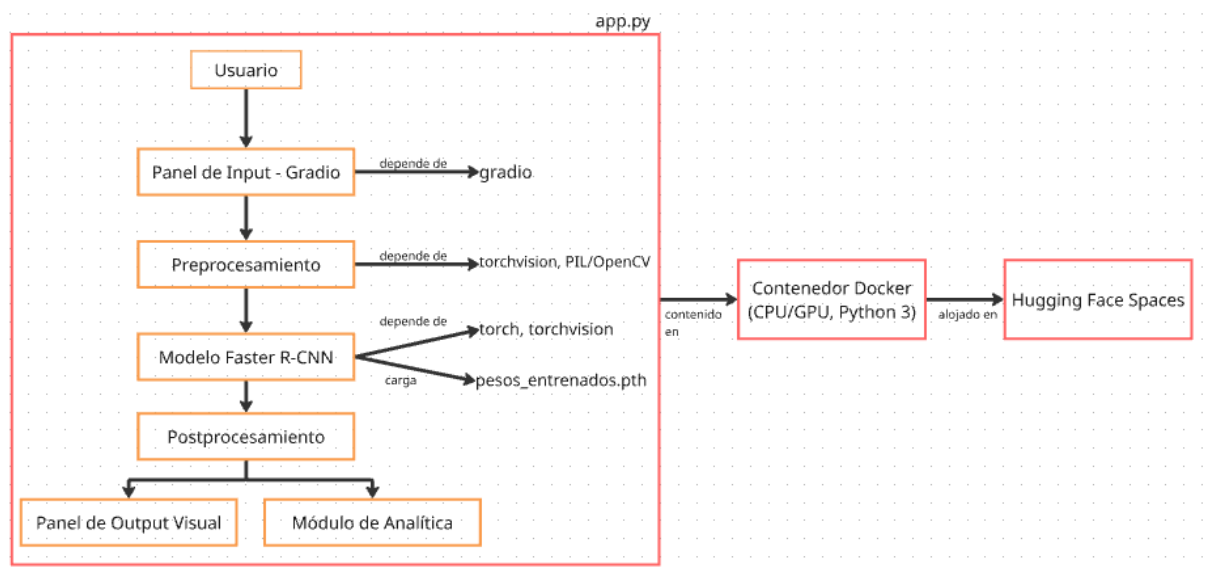
La arquitectura de ejecución e infraestructura de software sigue un flujo desacoplado pero eficiente:

- **Entorno de Ejecución (Runtime):** La aplicación corre sobre un contenedor Docker gestionado de forma nativa por Hugging Face, configurado con un entorno de CPU/GPU optimizado para Python 3.
- **Gestión del Modelo (Inferencia):** Al inicializarse el espacio, el script principal (app.py) levanta la arquitectura de Faster R-CNN con el backbone ResNet-50 + FPN y carga en memoria los pesos óptimos guardados (.pth) obtenidos al final de las 100 épocas de entrenamiento.
- **Pipeline de Inferencia:** Cuando un usuario carga una imagen, esta pasa de forma síncrona por las funciones de preprocesamiento

definitivas integradas en el pipeline (redimensionamiento a 640×640 píxeles y normalización según los parámetros de ImageNet). Posteriormente, el tensor es evaluado por el modelo en modo de evaluación (`model.eval()`), aplicando un umbral de confianza parametrizado para discriminar predicciones débiles antes de renderizar los resultados en la interfaz de Gradio.

3. Desarrollo del Software

Diagrama de componentes y dependencias



Guia de instalacion, configuracion y uso (README)

Requisitos previos

El proyecto fue desarrollado íntegramente en el entorno de Google Colab, las dependencias principales necesarias para replicar el entorno de inferencia son:

- Python 3.10+
- PyTorch 2.12.1 (con soporte CUDA 13.0 para aceleración por GPU)
- torchvision
- albumentations
- opencv-python (cv2)
- matplotlib

Estas pueden instalarse manualmente mediante:

```
pip install torch torchvision albumentations opencv-python matplotlib
```

Estructura del repositorio

El código fuente del proyecto se encuentra alojado en GitHub y está organizado de la siguiente manera:

- EDA_Videos_Vehiculos_Peru.ipynb: notebook con el análisis exploratorio de datos (Hito 1).
- Entrenamiento_01_Faster_RCNN.ipynb,
Entrenamiento_02_Faster_RCNN_mejorado.ipynb,
Entrenamiento_03_Faster_RCNN.ipynb: notebooks con las distintas iteraciones del proceso de entrenamiento del modelo.
- inferencia.py: script encargado de cargar el modelo entrenado y ejecutar predicciones sobre una imagen de entrada.

Los pesos del modelo entrenado (model.pt) no se almacenan directamente en el repositorio de GitHub debido a su tamaño, sino que se alojan de forma independiente en el Model Hub de Hugging Face, bajo el repositorio pollitoconpass/carritos-detectorV2.

Configuración

No se requiere configuración manual de variables de entorno ni descarga previa de archivos. El script inferencia.py obtiene automáticamente los pesos del modelo mediante la utilidad torch.hub.load_state_dict_from_url, la cual descarga el checkpoint directamente desde el repositorio de Hugging Face en tiempo de ejecución:

```
HF_REPO_ID = "pollitoconpass/carritos-detectorV2" hf_model_url =  
f"https://huggingface.co/{HF_REPO_ID}/resolve/main/model.pt"
```

El checkpoint incluye no solo los pesos del modelo, sino también el número de clases, las categorías (Carro, Mototaxi, Persona) y metadatos del entrenamiento (época óptima y mAP@50 alcanzado), los cuales se cargan automáticamente al ejecutar el script.

Uso

Existen dos formas de utilizar el modelo entrenado:

a) Inferencia local mediante script (inferencia.py)

Tras instalar las dependencias, se debe modificar la variable image_path dentro del script para apuntar a la imagen que se desea analizar, y ejecutar:

```
python inferencia.py
```

El script carga el modelo, procesa la imagen (redimensionamiento a 640×640 y normalización según parámetros de ImageNet), ejecuta la inferencia con un umbral de

confianza por defecto de 0.5, y muestra el resultado mediante matplotlib, dibujando las cajas delimitadoras coloreadas por clase (rojo para Carro, cian para Mototaxi, verde lima para Persona) junto con su respectivo score de confianza.

b) Uso mediante la aplicación web desplegada

Para una experiencia de uso sin necesidad de instalar dependencias, el modelo se encuentra desplegado como una aplicación interactiva en Hugging Face Spaces, accesible públicamente mediante el siguiente enlace:

<https://huggingface.co/spaces/pollitoconpapass/carritos-detector-ui>

4. Resultados y Conclusiones

Discusión de los resultados frente a la métrica objetivo

El objetivo principal del proyecto fue desarrollar un sistema confiable de detección de objetos capaz de identificar de manera precisa vehículos mayores, vehículos menores y peatones en el entorno urbano de Lima. Para evaluar su desempeño se utilizó como métrica principal el mAP@50 (Mean Average Precision con un umbral de IoU de 0.50).

Los resultados obtenidos en el conjunto de prueba alcanzaron un mAP@50 de 68.37% y un F1 macro de 64.66%. Estos valores muestran que la arquitectura Faster R-CNN con ResNet-50 + FPN ofrece un buen rendimiento para automatizar el monitoreo del tránsito. Uno de los factores más importantes para lograr este resultado fue la reducción de las 9 clases originales a solo 3 categorías principales (Carro, Mototaxi y Persona). Esta decisión simplificó el proceso de clasificación y redujo las confusiones entre vehículos con características similares, como buses, camiones y combis, que pasaron a formar parte de la categoría Carro.

Además, el modelo obtuvo un Recall de 91.2% en la clase Mototaxi, lo que indica que fue capaz de detectar la gran mayoría de estos vehículos en las imágenes evaluadas. Aunque esto redujo la precisión (46.4%) debido a la presencia de falsos positivos, en aplicaciones como el monitoreo del tránsito o la seguridad ciudadana resulta más conveniente detectar la mayor cantidad posible de vehículos, incluso si algunas detecciones son incorrectas, antes que dejar de detectar vehículos que realmente están presentes.

Limitaciones identificadas

- **Dependencia de la resolución y del tamaño de los objetos:** El rendimiento del modelo disminuye cuando los objetos, especialmente las personas, aparecen muy lejos de la cámara. Al ocupar pocos píxeles en la imagen, se pierde información importante sobre su forma, lo que aumenta la probabilidad de clasificarlos de manera incorrecta o de no detectarlos.

- **Ambigüedad causada por la perspectiva de la cámara:** Como el modelo trabaja con imágenes captadas por cámaras de seguridad fijas y ubicadas en altura, algunas mototaxis vistas desde atrás pueden parecerse a autos pequeños, especialmente cuando la iluminación es deficiente. Esto dificulta que el modelo pueda diferenciarlas correctamente.
- **Mayor tiempo de procesamiento:** Faster R-CNN realiza la detección en dos etapas, lo que permite obtener una localización más precisa de los objetos. Sin embargo, este proceso requiere más tiempo que los modelos de una sola etapa, por lo que puede resultar menos eficiente cuando se necesita procesar simultáneamente el video de muchas cámaras en tiempo real.

Propuestas de Trabajo futuro

- **Evaluar detectores modernos de una sola etapa (YOLOv8 o YOLOv10):** Como continuación del proyecto, se propone evaluar arquitecturas más recientes que permitan mantener o mejorar la precisión, especialmente en objetos pequeños, mientras reducen significativamente el tiempo de procesamiento para aplicaciones en tiempo real.
- **Incorporar la técnica SAHI:** Para mejorar la detección de peatones y vehículos que aparecen lejos de la cámara, se propone utilizar la técnica SAHI (Slicing Aided Hyper Inference). Esta técnica divide la imagen en secciones más pequeñas antes de realizar la detección, permitiendo identificar con mayor facilidad objetos de tamaño reducido sin necesidad de volver a entrenar el modelo.
- **Ampliar el conjunto de datos:** Se recomienda incorporar imágenes captadas en diferentes condiciones, como lluvia, neblina y escenas nocturnas. Asimismo, sería conveniente aprovechar videos urbanos sin etiquetar mediante técnicas de aprendizaje semisupervisado, con el fin de mejorar la capacidad del modelo para reconocer objetos en una mayor variedad de situaciones y reducir los errores de clasificación.

5. Anexos

Enlace al repositorio: <https://github.com/pollitoconpapass/carritos-peruanitos-detector>

Enlace al video demostrativo:

https://drive.google.com/drive/folders/1vrR_DDOcayvXOqTqoS045_Zd0ZYmj-q6?usp=sharing